Processamento de Sinal e Imagem
Desenvolvimento de um Filtro FIR

Aluno: Alexandre Saraiva nº44485

Docente: Prof. Dr. António Pinheiro

Disciplina: Processamento de Sinal e Imagem

Universidade da Beira Interior

Covilhã 2025

Índice

Lista de Figuras 3
Introdução 4
Componentes 5
Implementação do Circuito 6
Teste do Circuito 7
Programação Arduíno 2560 10
Resultados 15
Comparação 25
Conclusão 27

Lista de Figuras

Figura 1-Esquema Idêntico ao Implementado 4
Figura 2-Montagem e Componentes 5
Figura 3-Circuito 5
Figura 4-PINout DAC 6
Figura 5-PINout LF356 6
Figura 6-PINout Arduino ATmega 2560 6
Figura 7-Esquema DAC 6
Figura 8-Resposta em Frequência Filtro FIR teórico 9
Figura 9-Imagem Sinal com frequência 10Hz Obtida PicoScope7 15
Figura 10--Imagem Sinal com frequência 20Hz Obtida PicoScope7 15
Figura 11- Imagem Sinal com frequência 30Hz Obtida PicoScope7 16
Figura 12- Imagem Sinal com frequência 40Hz Obtida PicoScope7 16
Figura 13- Imagem Sinal com frequência 50Hz Obtida PicoScope7 17
Figura 14-Imagem Sinal com frequência 60Hz Obtida PicoScope7 17
Figura 15-Imagem Sinal com frequência 70Hz Obtida PicoScope7 18
Figura 16-Imagem Sinal com frequência 80Hz Obtida PicoScope7 18
Figura 17- Imagem Sinal com frequência 90Hz Obtida PicoScope7 19
Figura 18-Imagem Sinal com frequência 100Hz Obtida PicoScope7 19
Figura 19- Imagem Sinal com frequência 200Hz Obtida PicoScope7 20
Figura 20- Imagem Sinal com frequência 300Hz Obtida PicoScope7 20
Figura 21- Imagem Sinal com frequência 400Hz Obtida PicoScope7 21
Figura 22-Imagem Sinal com frequência 500Hz Obtida PicoScope7 21
Figura 23-Imagem Sinal com frequência 600Hz Obtida PicoScope7 22
Figura 24 -Imagem Sinal com frequência 700Hz Obtida PicoScope7 22
Figura 25-Imagem Sinal com frequência 800Hz Obtida PicoScope7 23
Figura 26- Imagem Sinal com frequência900Hz Obtida PicoScope7 23
Figura 27-Imagem representa a obtenção da frequência de Amostragem obtida no PicoScope7 24
Figura 28-Imagem Obtida Matlab representação gráfica dos dados obtidos 26

Introdução

Este trabalho prático laboratorial teve como propósito projetar e implementar um filtro FIR. Um filtro FIR (Finite Impulse Response) é um tipo de filtro digital amplamente utilizado no processamento de sinais devido à sua estabilidade garantida e à capacidade de operar com resposta de fase linear, preservando a forma dos sinais processados. A sua principal característica é que a resposta ao impulso tem duração finita, já que não utiliza realimentação na sua estrutura. A saída do filtro é obtida como uma soma ponderada das amostras de entrada, permitindo que ele seja projetado para atender a diversas especificações de filtragem, como rejeição de ruídos ou isolamento de determinadas frequências. Devido a essas propriedades, os filtros FIR encontram aplicação em sistemas de áudio, telecomunicações, controlo e processamento de imagens.

Neste trabalho laboratorial foi realizado a implementação de um filtro FIR de ordem N=25 e uma ωc normalizada de 0.002. Para implementar o filtro FIR, comecei por fazer um código em Assembly, onde configurei o ADC, de seguida montei o DAC de forma a garantir que x[n]=y[n]. Utilizei o Matlab para calcular os coeficientes bk e programar o filtro. Após a implementação, realizei medições para diferentes frequências, seguindo uma escala logarítmica.

O filtro FIR (Finite Impulse Response) é caracterizado por uma resposta ao impulso discreto h[n] de duração finita, que pode ser descrita pela equação de diferenças:

$$y[n] = h_0 x[n] + h_1 x[n-1] + \cdots + h_M x[n-M]$$

nesta equação, x[n]é o sinal de entrada, y[n] é o sinal de saída, e hM são os coeficientes do filtro. Também é possível expressá-la de forma simplificada como:

$$y[n] = \sum b_k \cdot x[n-k]$$

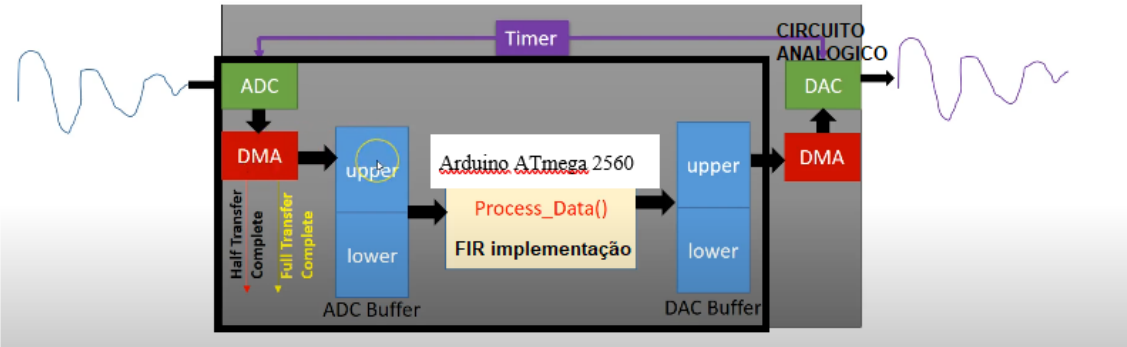

Figura 1-Esquema Idêntico ao Implementado

Componentes

-Breadboard

-(Osciloscópio) Software PicoScope 7;

− Fonte de Alimentação;

− Gerador de Sinais;

− 2Resistências;

− Amplificador Operacional;

− DAC 0832;

− Arduíno ATMega2560

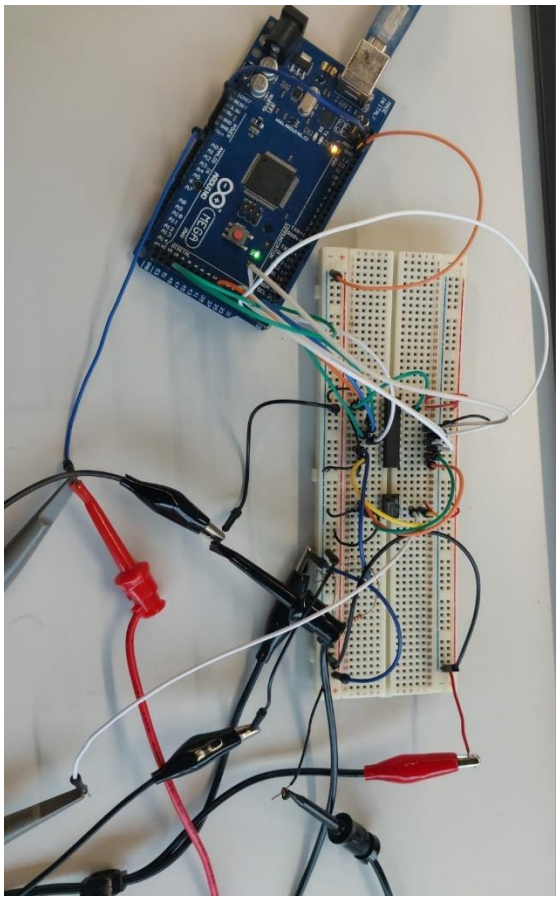

Figura 2-Montagem e Componentes

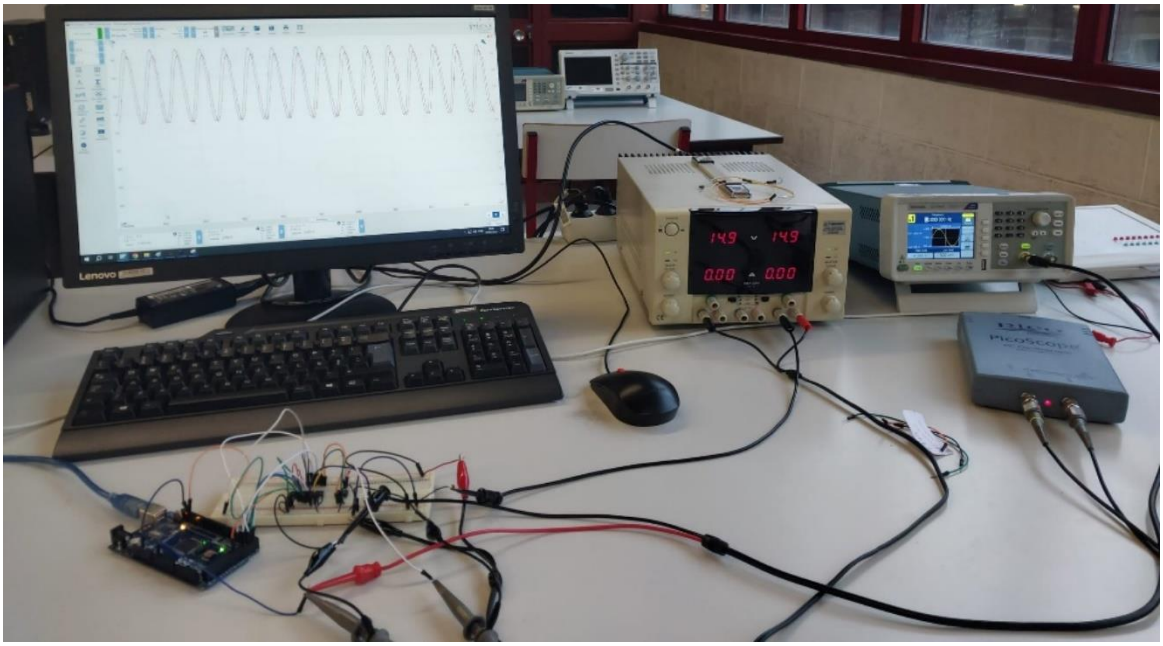

Figura 3-Circuito

Implementação do Circuito

Microcontrolador ATmega2560:

Fornece os sinais digitais e controla as operações no DAC0832.

Configura as portas de saída para controlar os pinos de dados e controlo do DAC0832.

DAC0832:

Converte os sinais digitais enviados pelo ATmega2560 em sinais analógicos (em corrente).

Sinais de controlo (CS, WR1, WR2, ILE e XFER).

O sinal analógico em corrente é processado pelo amplificador operacional para conversão corrente-tensão.

Amplificador Operacional LF356:

Configurado como amplificador inversor para conversão corrente-tensão.

A saída do amplificador fornece o sinal analógico final.

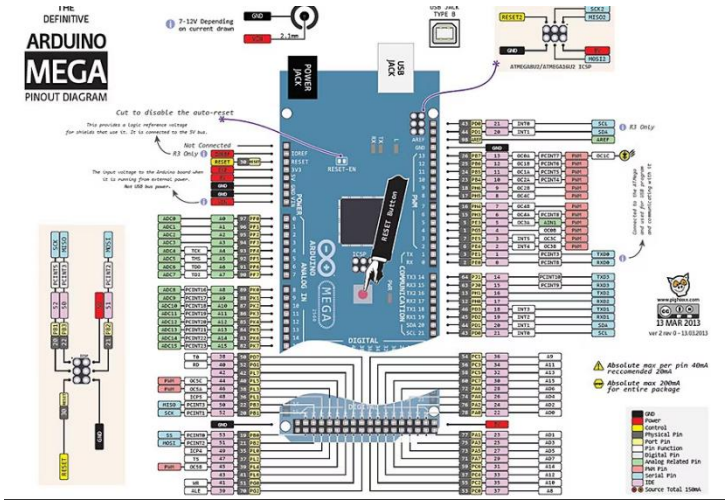

Figura 6-PINout Arduíno ATmega 2560

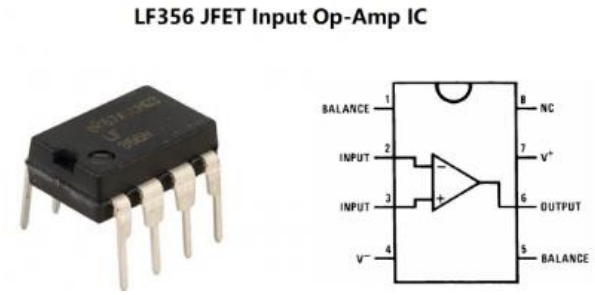

Figura 5-PINout LF356

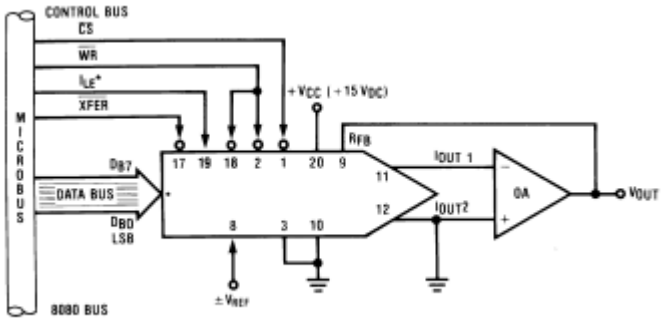

Figura 7-Esquema DAC

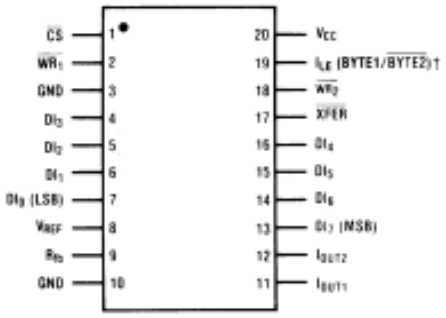

Figura 4-PINout DAC

Teste do Circuito

Para testar se o circuito estava a receber os sinais corretamente, coloquei o sinal de entrada igual ao sinal de saída x[n]=y[n], através de programação em Assembly, que basicamente configura o ADC para ler um sinal analógico (entrada) continuamente e enviar o valor convertido (sinal digital) para o PORTA., conectado ao DAC0832, converte o sinal digital de volta para analógico, permitindo a visualização do sinal de saída no osciloscópio (PicoScope7). Como o circuito estava a funcionar como pretendido e avancei para o cálculo dos Bks.

Programação em ASM do teste do Circuito:

```
.include "m2560def.inc" ; Inclui o arquivo de definição do microcontrolador
ATmega2560

; Salta para o ponto de início do programa
jmp INICIO

; Configuração para interrupção do ADC
.org 0x003A
jmp CONVERSAO ; Salta para a sub-rotina de conversão quando ocorrer uma
interrupção

; Início do programa
INICIO:
    ; Inicializa o Stack Pointer (ponteiro da pilha)
    ldi R16, LOW(RAMEND)
    out SPL, R16            ; Define a parte baixa do Stack Pointer
    ldi R16, HIGH(RAMEND)
    out SPH, R16            ; Define a parte alta do Stack Pointer

    ; Configuração das portas
    ldi R16, $FF
    out DDRA, R16           ; Define PORTA como saída (para enviar dados ao DAC)
    out DDRC, R16           ; Define PORTC como saída (controle do WR)

    ; Configuração inicial do PORTC
    sbi PORTC, 0            ; Define WR = 1 (não ativo, aguardando para ativar)

    ; Configuração do ADC
    ldi R16, $21            ;Entrada analógica usada: ADC1 (pino PA1), pino PA0
não funcionava
    sts ADMUX, R16          ; Configura o ADC para usar o canal 0 e referência
AVCC
    ldi R16, $CF
    sts ADCSRA, R16         ; Ativa o ADC (ADEN), inicia a conversão (ADSC),
                            ; habilita interrupções do ADC (ADIE) e define o
prescaler (mais amostras por segundo)

    sei                     ; Ativa interrupções globais

; Loop principal
LOOP:
    rjmp LOOP               ; Loop infinito para manter o programa ativo
    rjmp LOOP               ; (duplicado, mas funcionalmente equivalente a um
único rjmp LOOP)

; Sub-rotina de conversão do ADC
```

```
CONVERSAO:
    lds R16, ADCH            ; Lê o valor de 8 bits mais significativo do ADC
    out PORTA, R16           ; Envia o valor lido para PORTA (sinal digital
convertido)

    cbi PORTC, 0             ; Define WR = 0 (ativo, escreve o valor no DAC)
    sbi PORTC, 0             ; Define WR = 1 (não ativo, finaliza o pulso de
escrita)

    ldi R16, $CF
    sts ADCSRA, R16          ; Reativa o ADC: inicia a próxima conversão

    reti                     ; Retorna da interrupção
```

Código Matlab para gerar Bks:

```
%N corresponde à ordem do filtro
N=25;
%wc corresponde ao ripple
wc=0.002;
%Filtro FIR passa baixo
b=fir1(N,wc); %cálculo dos bk's
b1=b*256; %cálculo dos bk's deslocados
b2=round(b1);
%Dimensionar o vetor
dim=1:1:numel(b);
tab=[dim',b',b2']
```

Tabela com o valor dos Bks gerado através do Matlab:

b[k]	Originais	Convertidos
1.0	0.0059	2.0
2.0	0.007	2.0
3.0	0.0101	3.0
4.0	0.0151	4.0
5.0	0.0216	6.0
6.0	0.0293	7.0
7.0	0.0376	10.0
8.0	0.0461	12.0
9.0	0.0542	14.0
10.0	0.0614	16.0
11.0	0.0672	17.0
12.0	0.0713	18.0
13.0	0.0734	19.0
14.0	0.0734	19.0
15.0	0.0713	18.0
16.0	0.0672	17.0
17.0	0.0614	16.0
18.0	0.0542	14.0
19.0	0.0461	12.0
20.0	0.0376	10.0
21.0	0.0293	7.0
22.0	0.0216	6.0
23.0	0.0151	4.0
24.0	0.0101	3.0
25.0	0.007	2.0
26.0	0.0059	2.0

Resposta em Frequência do filtro FIR implementado teoricamente

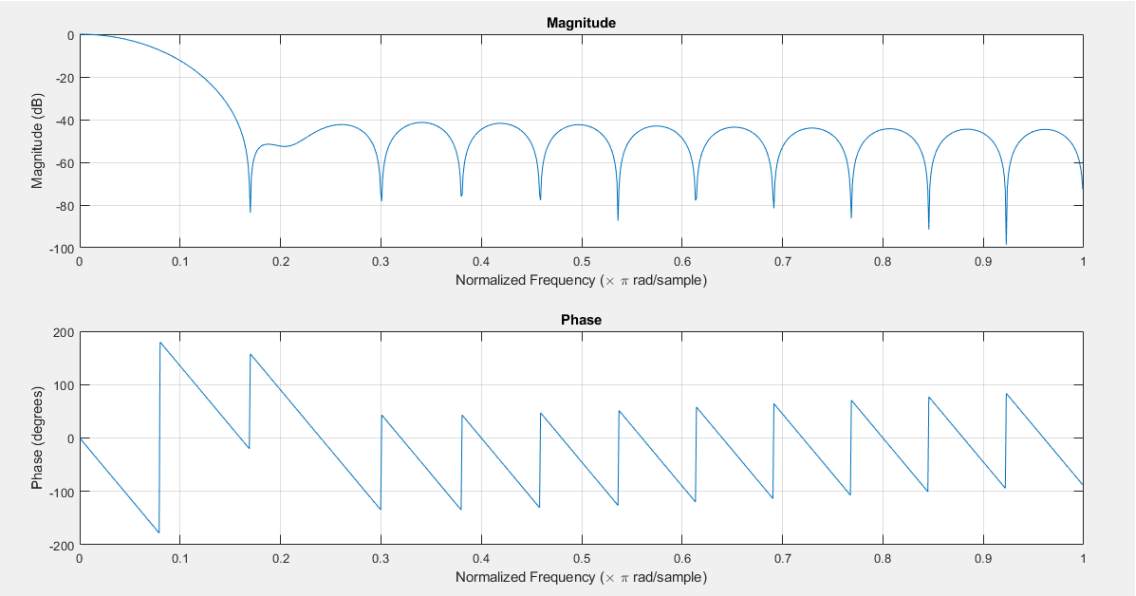

Figura 8-Resposta em Frequência Filtro FIR teórico

Código Matlab para implementar a resposta em Frequência do Filtro FIR:

```
% Definição dos parâmetros
N = 25; % Ordem do filtro
wc = 0.002; % Fator de corte (normalizado)

% Criação do filtro FIR passa-baixo
b = fir1(N, wc); % Coeficientes do filtro

% Cálculo da resposta em frequência
[H, w] = freqz(b, 1, 1024); % Resposta em frequência

% Plot da resposta em magnitude e fase
figure;

% Magnitude
subplot(2, 1, 1);
plot(w / pi, 20*log10(abs(H))); % Normaliza frequência
grid on;
title('Magnitude');
xlabel('Normalized Frequency (\times \pi rad/sample)');
ylabel('Magnitude (dB)');

% Fase
subplot(2, 1, 2);
plot(w / pi, angle(H) * (180 / pi)); % Converte fase para graus
grid on;
title('Phase');
xlabel('Normalized Frequency (\times \pi rad/sample)');
ylabel('Phase (degrees)');
```

Programação Arduíno 2560

```
.include "m2560def.inc" ; Inclui as definições do ATmega2560

; Declaração de endereços para armazenar dados
.equ data_R10=0x0201 ; Endereço para armazenar X[n-1]
.equ data_R11=0x0202 ; Endereço para armazenar X[n-2]
.equ data_R12=0x0203 ; Endereço para armazenar X[n-3]
.equ data_R13=0x0204 ; Endereço para armazenar X[n-4]
.equ data_R14=0x0205 ; Endereço para armazenar X[n-5]
.equ data_R15=0x0206 ; Endereço para armazenar X[n-6]
.equ data_R16=0x0207 ; Endereço para armazenar X[n-7]
.equ data_R18=0x0201 ; Endereço para armazenar X[n-9]
.equ data_R19=0x0203 ; Endereço para armazenar X[n-10]
.equ data_R20=0x0204 ; Endereço para armazenar X[n-11]
.equ data_R21=0x0205 ; Endereço para armazenar X[n-12]

jmp inicio ; Salta para o início do programa

; Configuração da interrupção
.org 0x003A ; Define o vetor de interrupção para o ADC
jmp converter ; Salta para a rotina de conversão

; Inicialização do programa
inicio:
    ldi R16, LOW(RAMEND) ; Configura o ponteiro da pilha (parte baixa)
    out SPL, R16
    ldi R16, HIGH(RAMEND) ; Configura o ponteiro da pilha (parte alta)
    out SPH, R16 ; Inicialização da stack (pilha)

    ldi R16, 0xFF
    out DDRA, R16 ; Define PORTA como saída (8 bits)
    out DDRC, R16 ; Define PORTC como saída (controle do WR)

    ldi R18, 0x03
    out PORTC, R18 ; Configura PORTC para WR inativo (nível alto)

    ldi R16, 0x21 ; Configura o ADC (referência AREF externa, canal ADC1)
    sts ADMUX, R16

    ldi R16, 0xCF ; Ativa o ADC, inicia conversão e habilita interrupções
    sts ADCSRA, R16

    clr R16 ; Zera o registrador R16 para inicialização dos dados
    ldi R16, 0x00
    ; Inicializa os endereços de memória dos dados
    sts data_R10, R16
    sts data_R11, R16
    sts data_R12, R16
    sts data_R13, R16
    sts data_R14, R16
    sts data_R15, R16
    sts data_R16, R16
    sts data_R18, R16
    sts data_R19, R16
    sts data_R20, R16
    sts data_R21, R16

    clr R16 ; Garante que R16 está zerado

    sei ; Habilita interrupções globais
```

```
loop:
    rjmp loop ; Loop infinito para manter o programa em execução

; Rotina de interrupção do ADC
converter:
    lds R16, ADCH ; Lê o valor convertido do ADC (8 bits mais significativos)
    rcall fconverter ; Chama a função de cálculo com Bk e X
    ldi R16, 0xCF ; Reativa o ADC para a próxima conversão
    sts ADCSRA, R16
    reti ; Retorna da interrupção

; Função de cálculo dos coeficientes Bk e entrada X
fconverter:
    clr R4 ; Inicializa acumuladores para o cálculo
    clr R3
    clr R30

    clr R17 ; Zera os registos temporários
    clr R1
    clr R0

    ; Início do cálculo dos coeficientes Bk e entradas X
    ldi R17, 2 ; Bk=1
    mul R16, R17 ; Multiplica X[n] por Bk=2
    add R3, R0
    adc R4, R1

    ldi R17, 2 ; Bk=2
    lds R30, data_R10 ; X[n-1]
    mul R30, R17 ; Multiplica X[n-1] por Bk=2
    add R3, R0
    adc R4, R1

    ldi R17, 3 ; Bk=3
    lds R30, data_R11 ; X[n-2]
    mul R30, R17 ; Multiplica X[n-2] por Bk=3
    add R3, R0
    adc R4, R1

    ldi R17, 4 ; Bk=4
    lds R30, data_R12 ; X[n-3]
    mul R30, R17
    add R3, R0
    adc R4, R1

    ldi R17, 6 ; Bk=5
    lds R30, data_R13 ; X[n-4]
    mul R30, R17
    add R3, R0
    adc R4, R1

    ldi R17, 7 ; Bk=6
    lds R30, data_R14 ; X[n-5]
    mul R30, R17
    add R3, R0
    adc R4, R1

    ldi R17, 10 ; Bk=7
    lds R30, data_R15 ; X[n-6]
    mul R30, R17
    add R3, R0
    adc R4, R1
```

```
ldi R17, 12 ; Bk=8
lds R30, data_R16 ; X[n-7]
mul R30, R17
add R3, R0
adc R4, R1

ldi R17, 14 ; Bk=9
lds R30, data_R18 ; X[n-9]
mul R30, R17
add R3, R0
adc R4, R1

ldi R17, 16 ; Bk=10
lds R30, data_R19 ; X[n-10]
mul R30, R17
add R3, R0
adc R4, R1

ldi R17, 17 ; Bk=11
lds R30, data_R20 ; X[n-11]
mul R30, R17
add R3, R0
adc R4, R1

ldi R17, 18 ; Bk=12
lds R30, data_R21 ; X[n-12]
mul R30, R17
add R3, R0
adc R4, R1

ldi R17, 19 ; Bk=13
mul R28, R17 ; Multiplica X[n-13] por Bk=19
add R3, R0
adc R4, R1

ldi R17, 19 ; Bk=14
mul R27, R17 ; Multiplica X[n-14] por Bk=19
add R3, R0
adc R4, R1

ldi R17, 18 ; Bk=15
mul R26, R17 ; Multiplica X[n-15] por Bk=18
add R3, R0
adc R4, R1

ldi R17, 17 ; Bk=16
mul R25, R17 ; Multiplica X[n-16] por Bk=17
add R3, R0
adc R4, R1

ldi R17, 16 ; Bk=17
mul R24, R17 ; Multiplica X[n-17] por Bk=16
add R3, R0
adc R4, R1

ldi R17, 14 ; Bk=18
mul R23, R17 ; Multiplica X[n-18] por Bk=14
add R3, R0
adc R4, R1

ldi R17, 12 ; Bk=19
```

```
    mul R22, R17 ; Multiplica X[n-19] por Bk=12
    add R3, R0
    adc R4, R1

    ldi R17, 10 ; Bk=20
    mul R21, R17 ; Multiplica X[n-20] por Bk=10
    add R3, R0
    adc R4, R1

    ldi R17, 7 ; Bk=21
    mul R20, R17 ; Multiplica X[n-21] por Bk=7
    add R3, R0
    adc R4, R1

    ldi R17, 6 ; Bk=22
    mul R19, R17 ; Multiplica X[n-22] por Bk=6
    add R3, R0
    adc R4, R1

    ldi R17, 4 ; Bk=23
    mul R18, R17 ; Multiplica X[n-23] por Bk=4
    add R3, R0
    adc R4, R1

    ldi R17, 3 ; Bk=24
    mul R15, R18 ; Multiplica X[n-24] por Bk=3
    add R3, R0
    adc R4, R1

    ldi R17, 2 ; Bk=25
    mul R14, R17 ; Multiplica X[n-25] por Bk=2
    add R3, R0
    adc R4, R1

    ldi R17, 2 ; Bk=26
    mul R13, R17 ; Multiplica X[n-26] por Bk=2
    add R3, R0
    adc R4, R1

; Atualização das variáveis X
    mov R13, R14 ; X[n-25]
    mov R14, R15 ; X[n-24]
    mov R15, R18 ; X[n-23]
    mov R18, R19 ; X[n-21]
    mov R19, R20 ; X[n-20]
    mov R20, R21 ; X[n-19]
    mov R21, R22 ; X[n-18]
    mov R22, R23 ; X[n-17]
    mov R23, R24 ; X[n-16]
    mov R24, R25 ; X[n-15]
    mov R25, R26 ; X[n-14]
    mov R26, R27 ; X[n-13]
    mov R27, R28
    lds R16, data_R21 ; X[n-12]
    mov R28, R16

    ; Atualiza os valores armazenados em memória
    lds R16, data_R20 ; X[n-11]
    sts data_R21, R16
    lds R16, data_R19 ; X[n-10]
    sts data_R20, R16
    lds R16, data_R18 ; X[n-9]
```

```
sts data_R19, R16
lds R16, data_R16 ; X[n-8]
sts data_R18, R16
lds R16, data_R15 ; X[n-7]
sts data_R16, R16
lds R16, data_R14 ; X[n-6]
sts data_R15, R16
lds R16, data_R13 ; X[n-5]
sts data_R14, R16
lds R16, data_R12 ; X[n-4]
sts data_R13, R16
lds R16, data_R11 ; X[n-3]
sts data_R12, R16
lds R16, data_R10 ; X[n-2]
sts data_R11, R16
lds R16, ADCH ; X[n-1]
sts data_R10, R16

; Envia o resultado acumulado ao PORTA
out PORTA, R4
ldi R18, 0x01 ; Ativa WR
out PORTC, R18
ldi R18, 0x00 ; Desativa WR
out PORTC, R18
ldi R18, 0x03 ; Configura WR como inativo novamente
out PORTC, R18
ret ; Retorna da função
```

Resultados

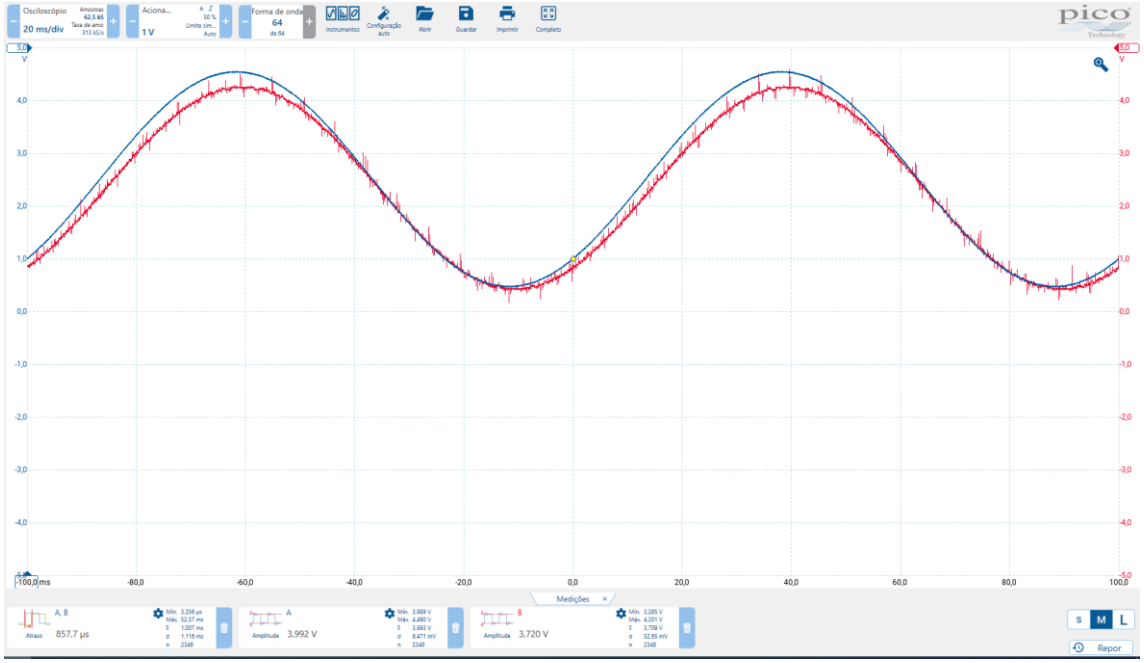

Figura 9-Imagem Sinal com frequência 10Hz Obtida PicoScope7

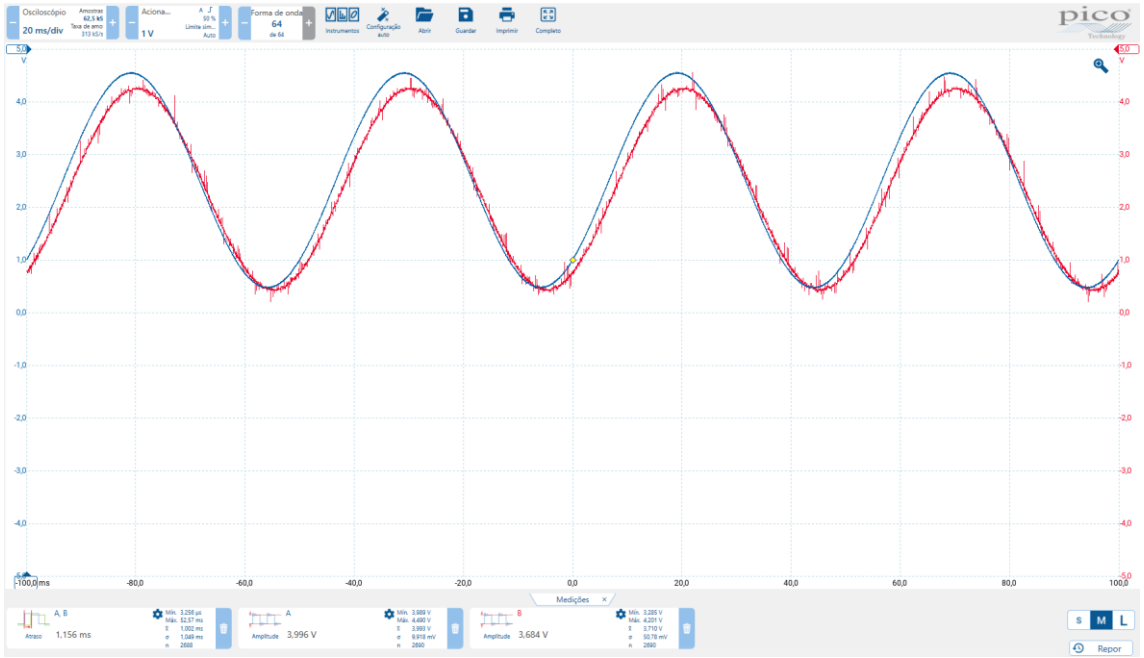

Figura 10--Imagem Sinal com frequência 20Hz Obtida PicoScope7

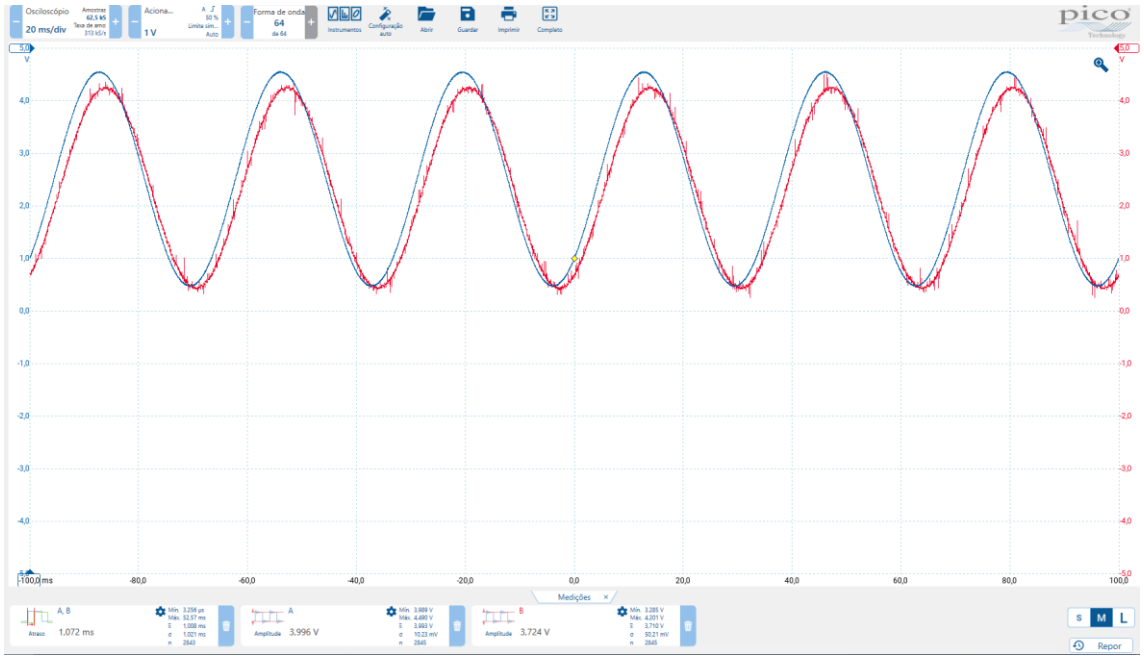

Figura 11- Imagem Sinal com frequência 30Hz Obtida PicoScope7

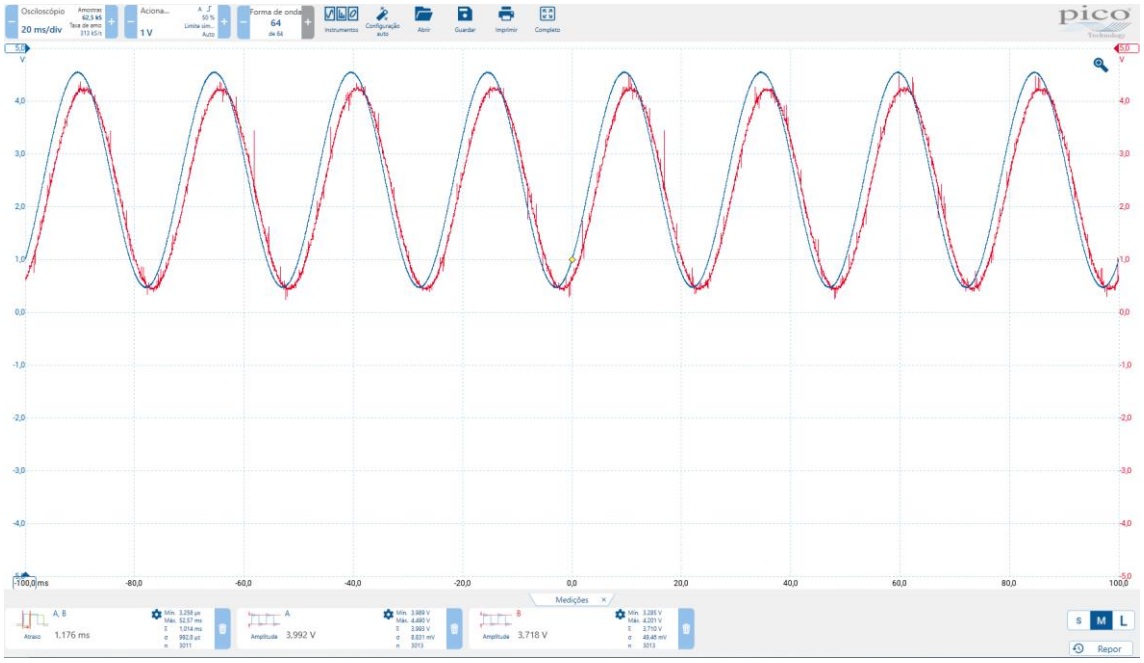

Figura 12- Imagem Sinal com frequência 40Hz Obtida PicoScope7

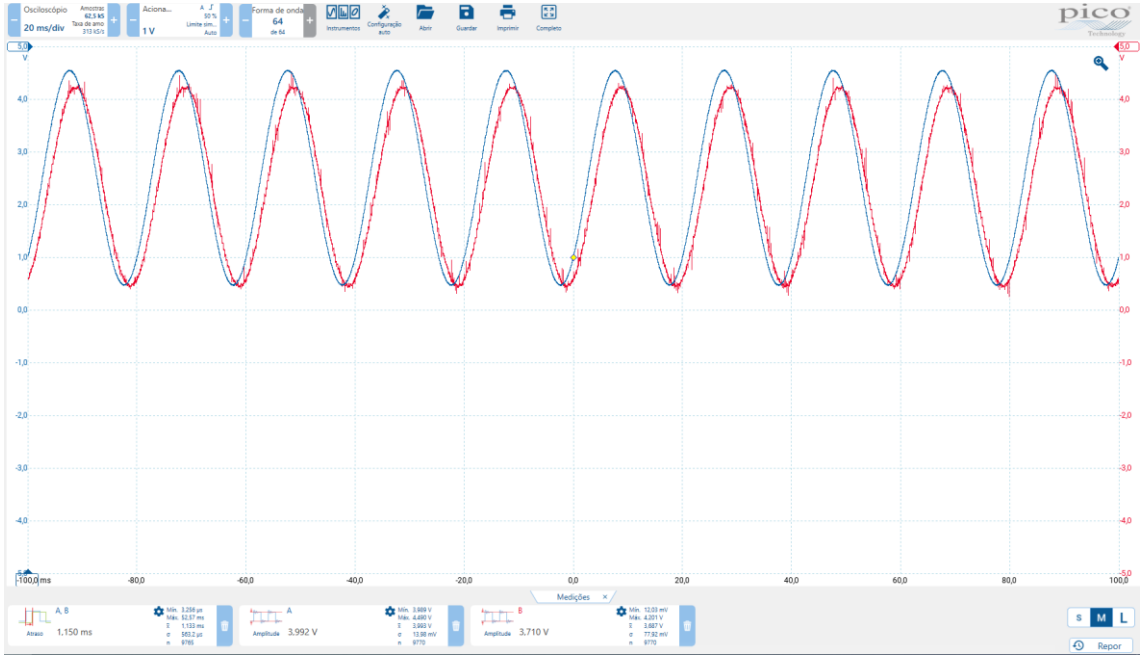

Figura 13- Imagem Sinal com frequência 50Hz Obtida PicoScope7

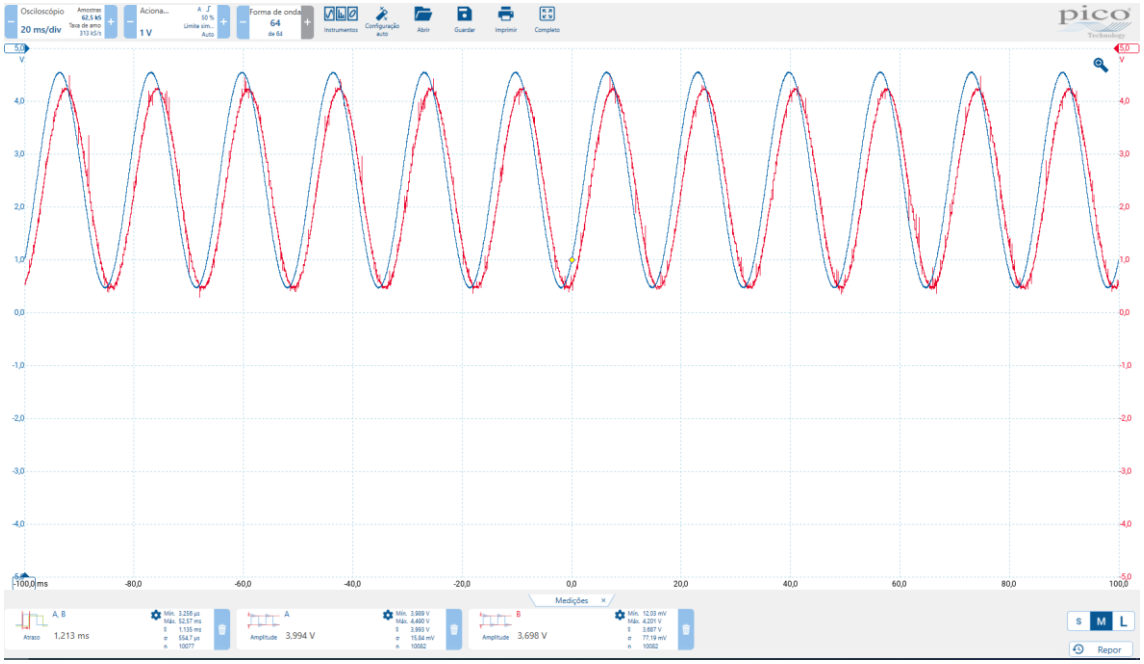

Figura 14-Imagem Sinal com frequência 60Hz Obtida PicoScope7

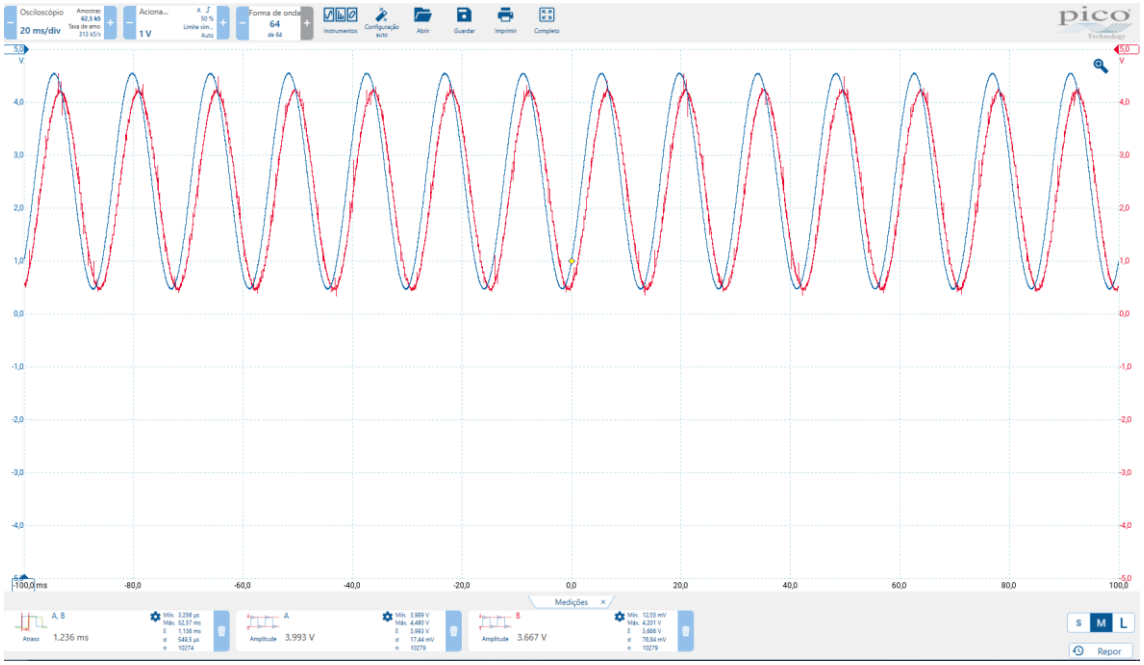

Figura 15-Imagem Sinal com frequência 70Hz Obtida PicoScope7

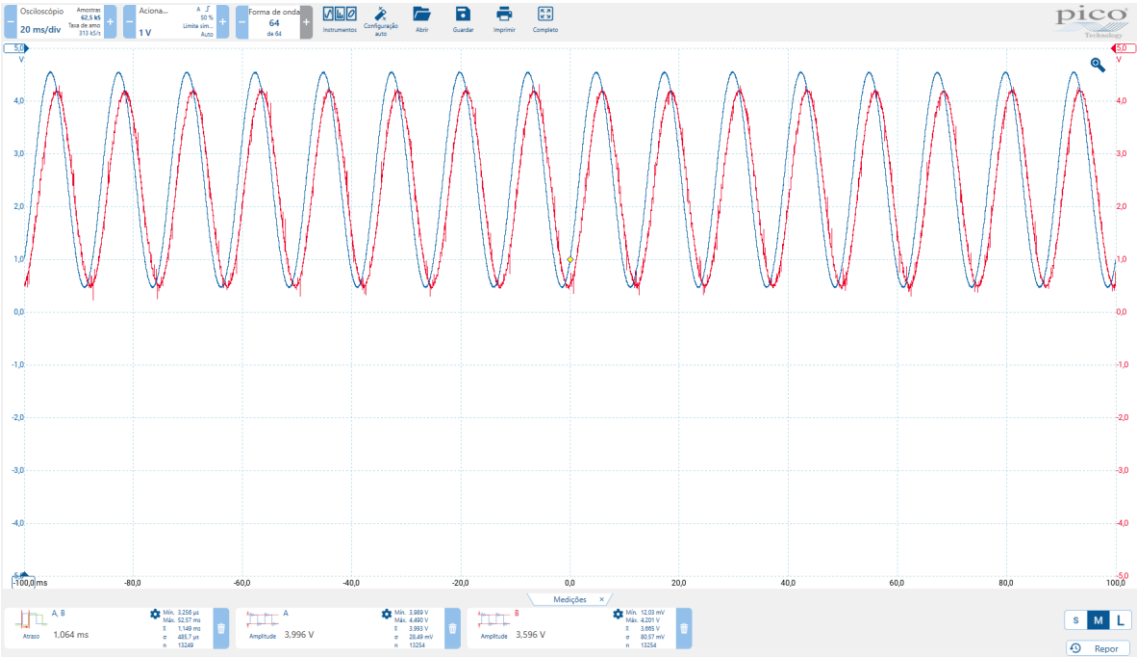

Figura 16-Imagem Sinal com frequência 80Hz Obtida PicoScope7

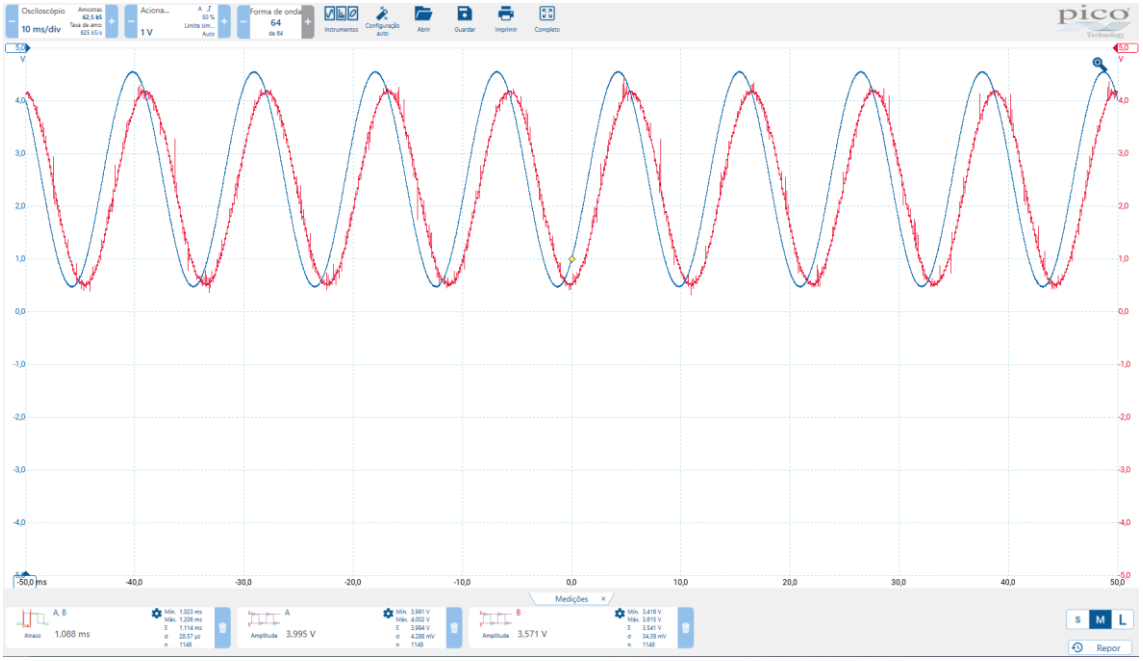

Figura 17- Imagem Sinal com frequência 90Hz Obtida PicoScope7

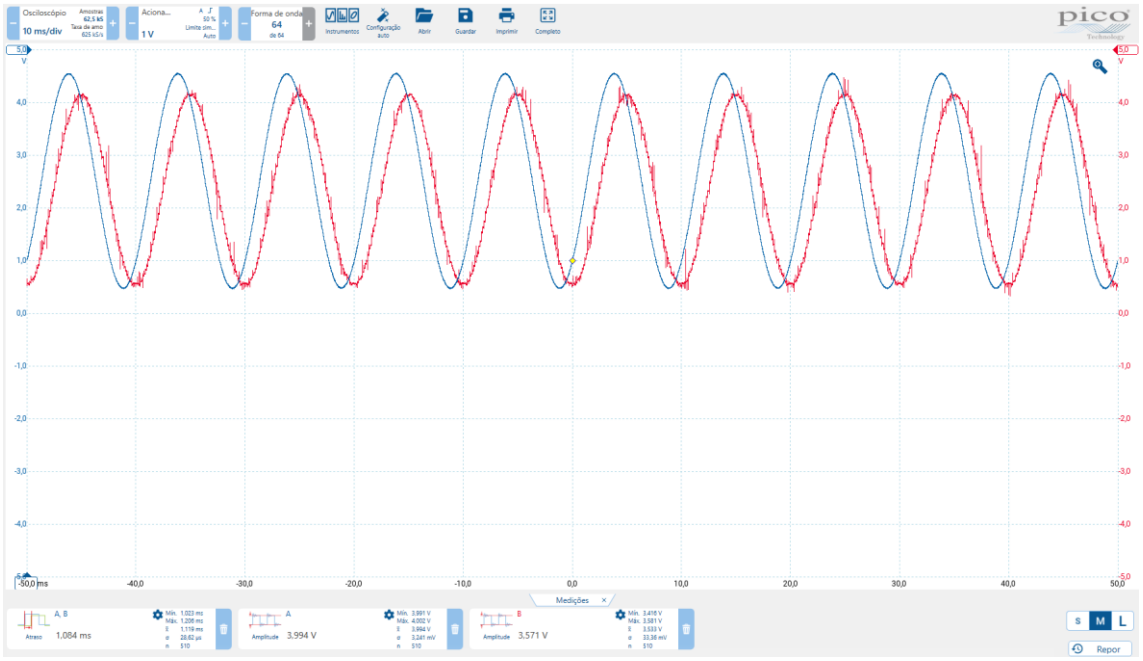

Figura 18-Imagem Sinal com frequência 100Hz Obtida PicoScope7

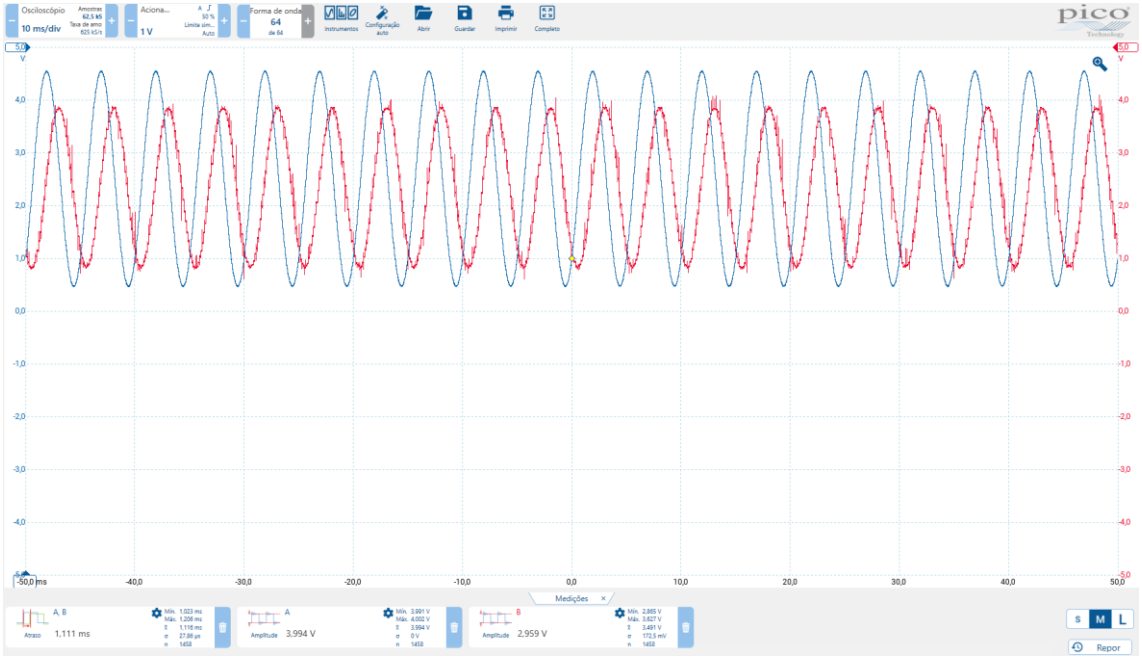

Figura 19- Imagem Sinal com frequência 200Hz Obtida PicoScope7

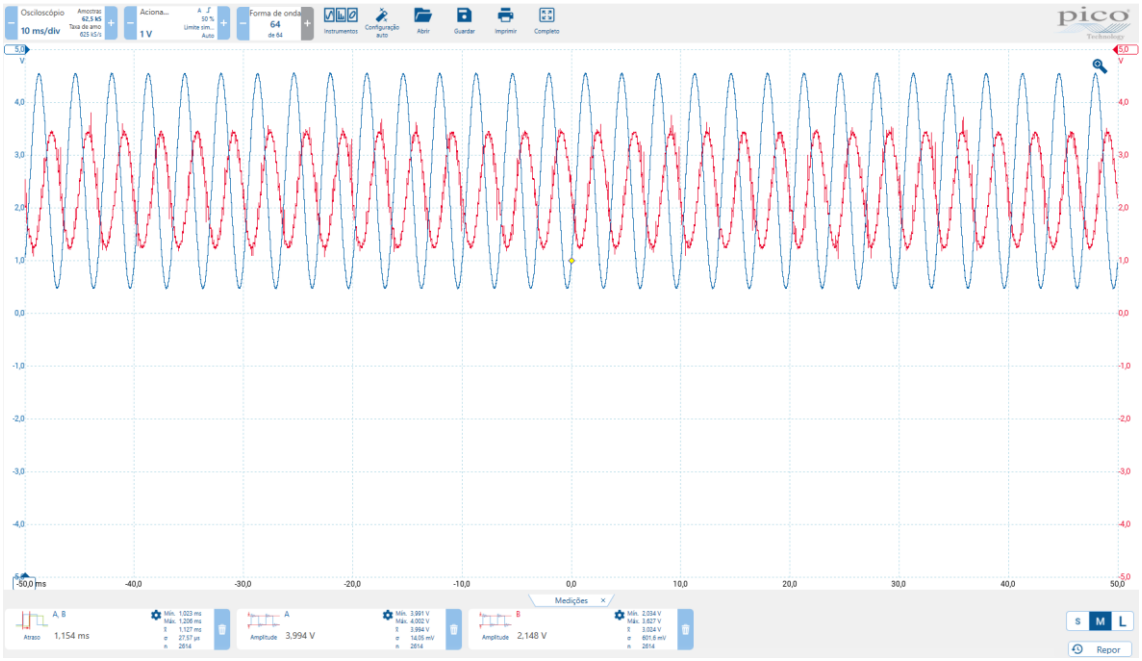

Figura 20- Imagem Sinal com frequência 300Hz Obtida PicoScope7

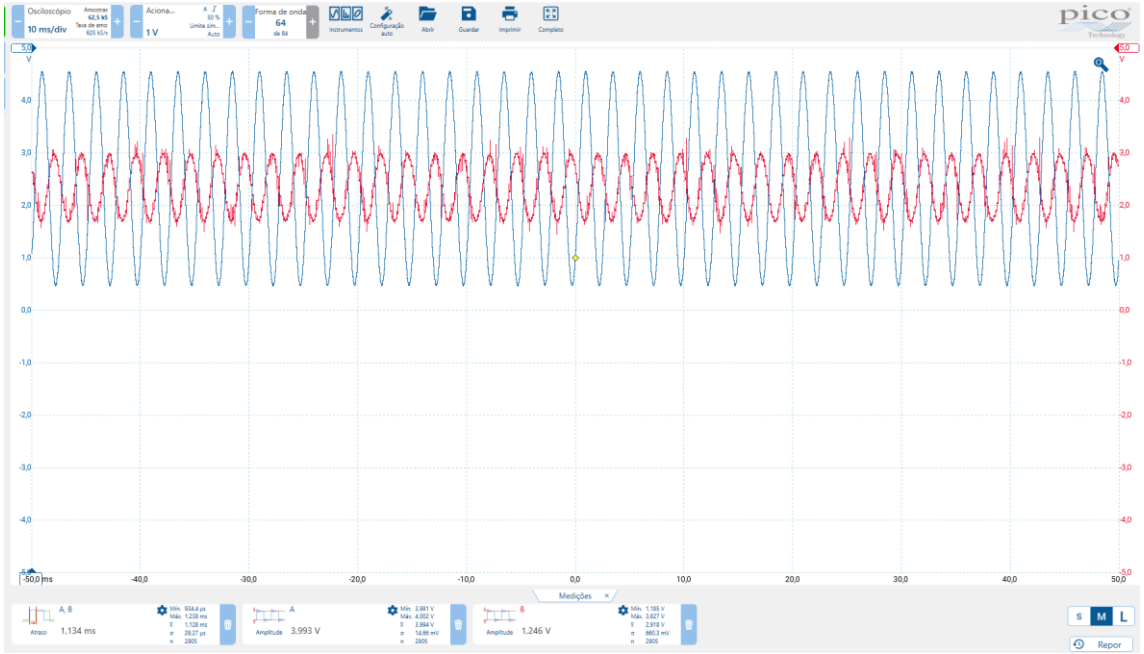

Figura 21- Imagem Sinal com frequência 400Hz Obtida PicoScope7

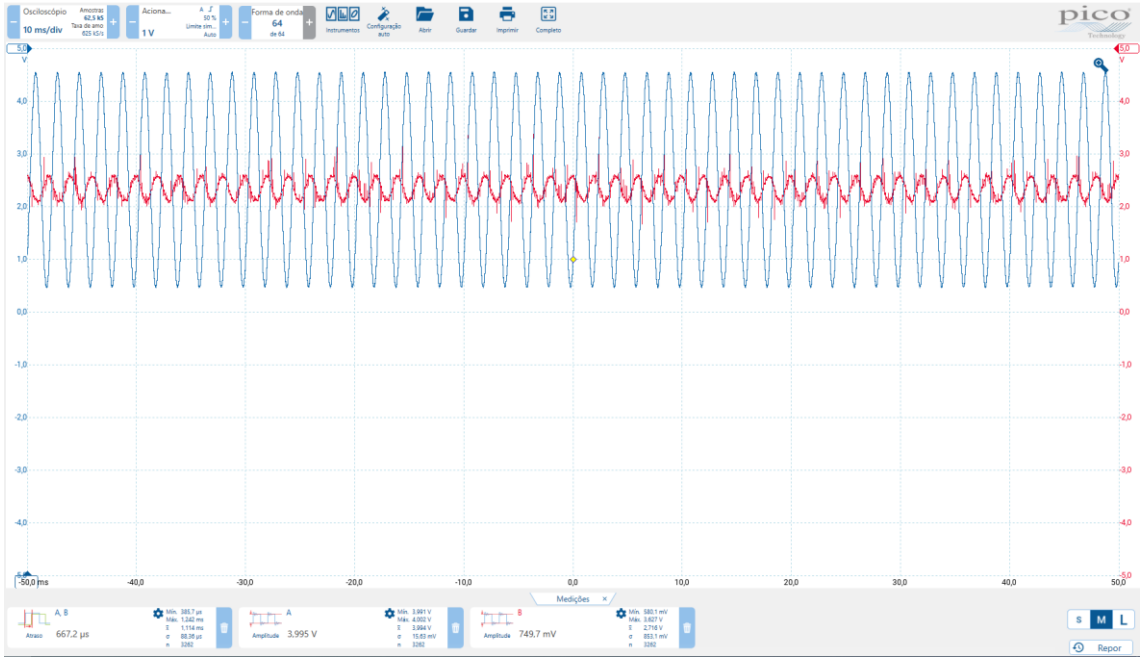

Figura 22-Imagem Sinal com frequência 500Hz Obtida PicoScope7

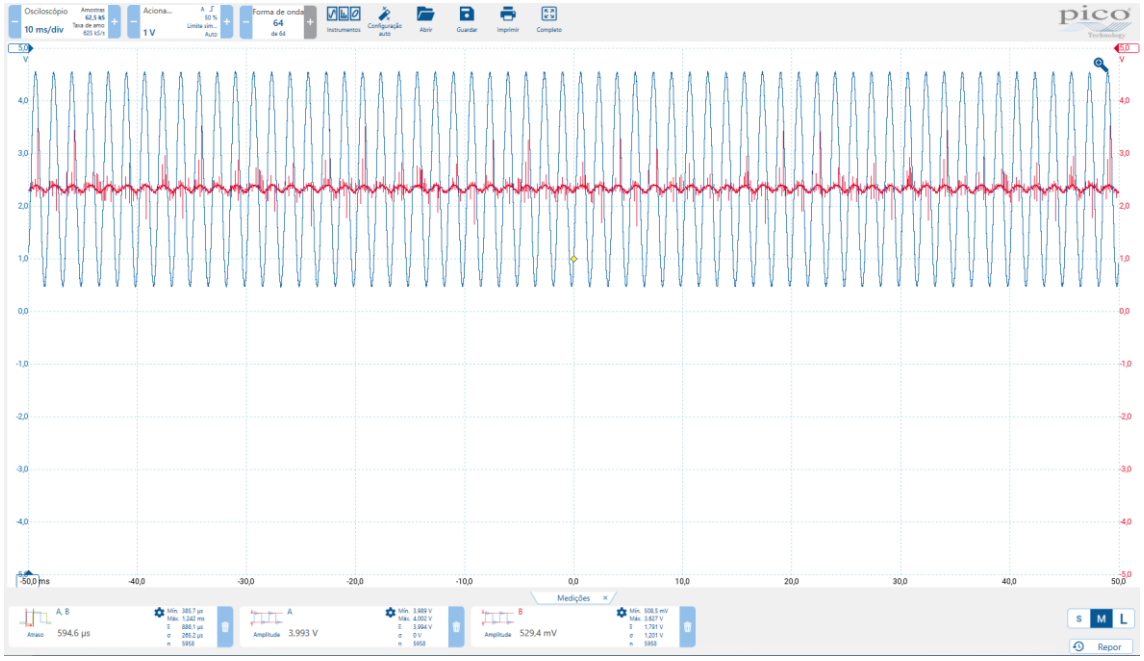

Figura 23-Imagem Sinal com frequência 600Hz Obtida PicoScope7

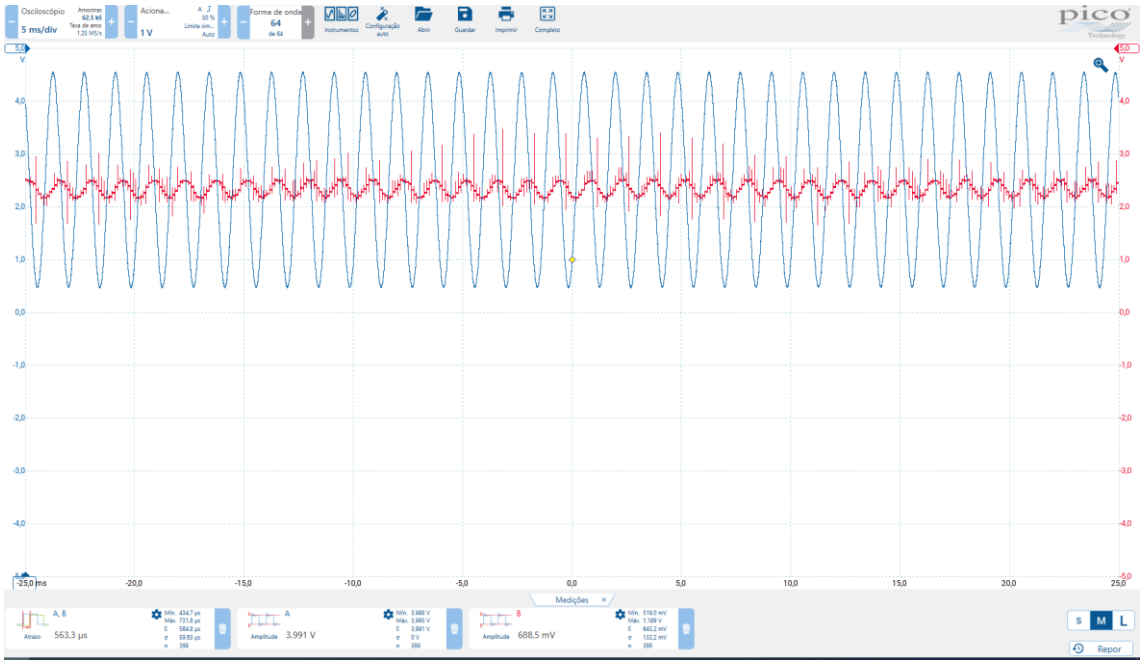

Figura 24 -Imagem Sinal com frequência 700Hz Obtida PicoScope7

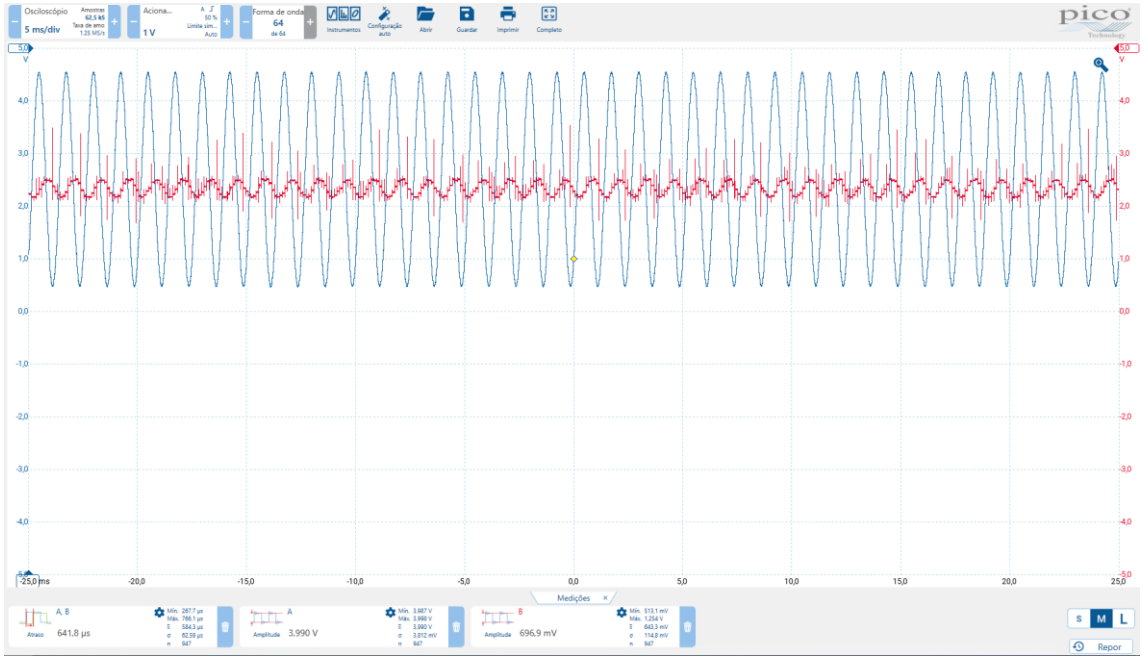

Figura 25-Imagem Sinal com frequência 800Hz Obtida PicoScope7

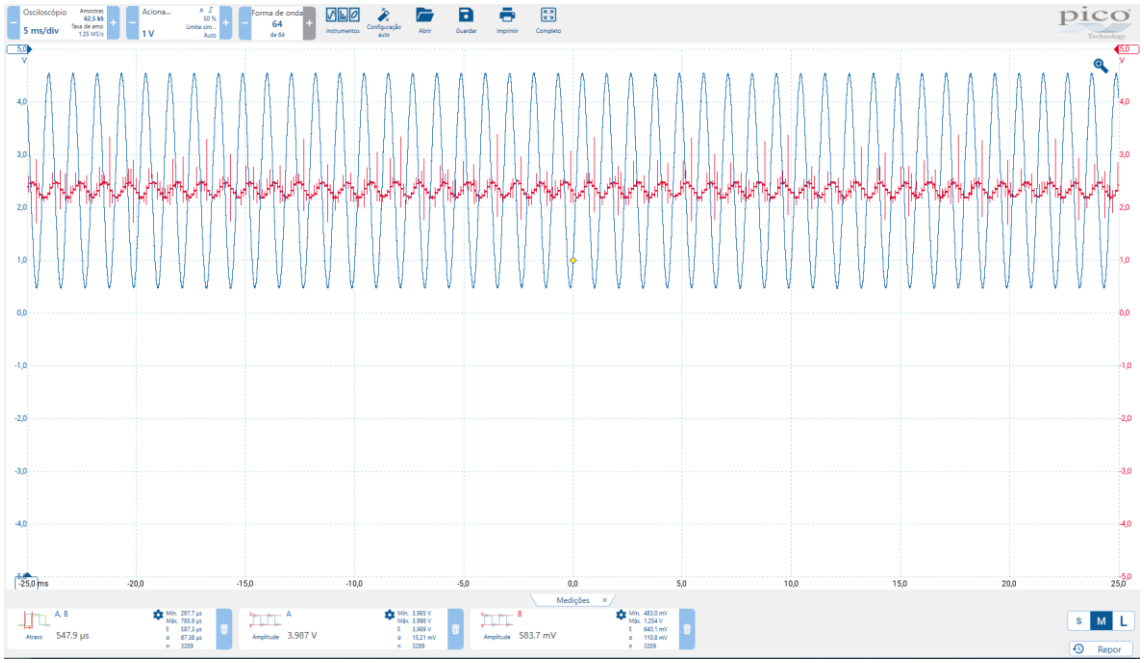

Figura 26- Imagem Sinal com frequência900Hz Obtida PicoScope7

Determinação da Frequência de Amostragem

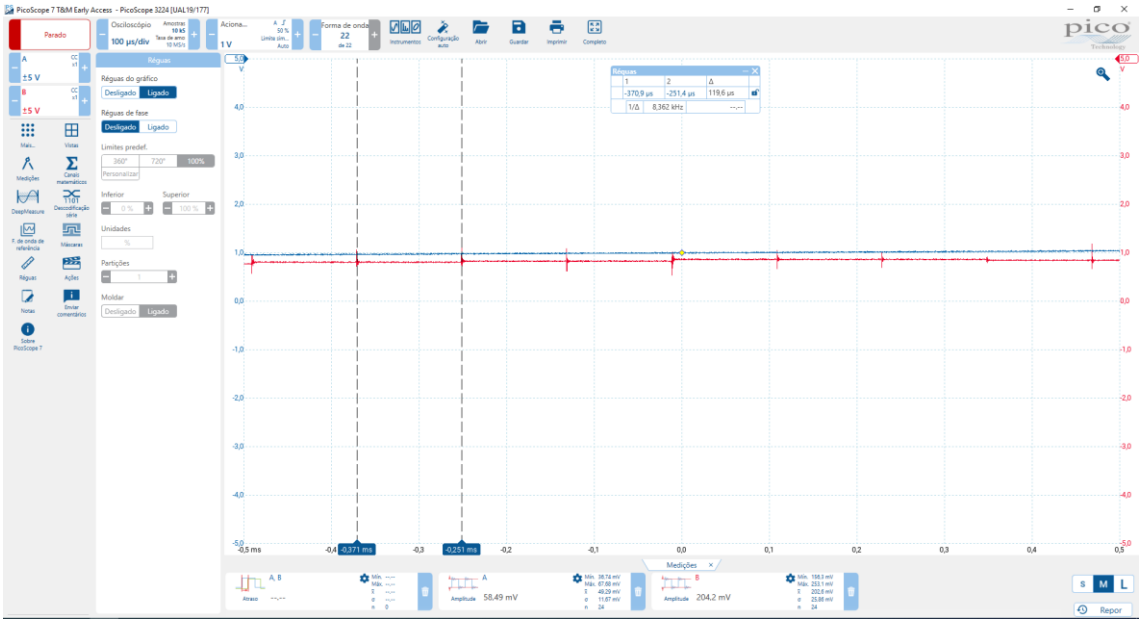

Figura 27-Imagem representa a obtenção da frequência de Amostragem obtida no PicoScope7

Frequência de Amostragem: 8362Hz

Tabela com valores retirados com o auxílio do PicoScope7

Frequência (Hz)	Vi (V)	Vo (V)	Δt (A)
10.0	3.99	3.709	-0.001
20.0	3.99	3.71	-0.001
30.0	3.99	3.71	-0.001
40.0	3.99	3.71	-0.001
50.0	3.99	3.687	-0.0011
60.0	3.99	3.687	-0.0011
70.0	3.99	3.686	-0.0011
80.0	3.99	3.678	-0.0011
90.0	3.99	3.541	-0.0011
100.0	3.99	3.534	-0.0011
200.0	3.99	3.491	-0.0011
300.0	3.99	3.024	-0.0011
400.0	3.99	2.918	-0.0011
500.0	3.99	2.718	-0.0011
600.0	3.99	1.825	0.0009
700.0	3.99	0.693	0.0005
800.0	3.99	0.643	0.0005
900.0	3.99	0.642	0.0006
1000.0	3.99	0.636	0.0006

Comparação

Comparação dos valores teóricos com valores experimentais.

Código em Matlab usado para a representação dos valores experimentais retirados:

```
clear all   % Remove todas as variáveis do workspace
close all   % Fecha todas as janelas de figuras abertas
clc         % Limpa a janela do console

% N corresponde à ordem do filtro
N=25;       % Define a ordem do filtro FIR

% wc corresponde ao ripple
wc=0.002;   % Frequência de corte normalizada do filtro

% Frequência de amostragem
fa=8362;    % Frequência de amostragem do sistema em Hz

% Projeto do filtro FIR
b=fir1(N,wc);      % Gera os coeficientes do filtro FIR usando janela de
Hamming
B1=b*256;          % Escala os coeficientes para melhorar precisão
(multiplicando por 256)
b1=round(B1);      % Arredonda os coeficientes para números inteiros

% Resposta em frequência do filtro FIR
[H,w]= freqz(B1,256,1024); % Calcula a resposta em frequência do filtro FIR
f1 = (w*fa);               % Converte a frequência normalizada em rad/s para
Hz

% Plot da resposta do filtro FIR (ganho e fase)
figure(1), freqz(B1,256,1024), grid on; % Plota a resposta em frequência
(ganho e fase) com grade

% Dados experimentais
fr = [10, 20, 30, 40, 50, 60, 70, 80, 90, 100, 200, 300, 400, 500, 600, 700,
800, 900, 1000];
% Frequências medidas em Hz

Vi = [3.99] * 19;  % Tensão de entrada constante experimental (multiplicação
usada para escala)
Vo = [3.71, 3.71, 3.71, 3.71, 3.69, 3.69, 3.69, 3.68, 3.54, 3.53, 3.49, 3.02,
2.92, 2.72, 1.82, 0.69, 0.64, 0.64, 0.64];
% Tensão de saída medida experimentalmente para diferentes frequências

deltaT = [-0.0010, -0.0010, -0.0010, -0.0010, -0.0011, -0.0011, -0.0011, -
0.0011, -0.0011, -0.0011, -0.0011, -0.0011, -0.0011, -0.0011, 0.0009, 0.0005,
0.0005, 0.0006, 0.0006];
% Diferença de tempo medida entre entrada e saída para calcular atraso de
fase

% Cálculo do ganho e da fase
Hdb = 20*log(H);               % Converte a resposta em frequência para ganho
em dB
hmed = 20*log(Vo./Vi);         % Calcula o ganho experimental em dB
(20*log10(Vout/Vin))
phimed = 360*fr.*deltaT;       % Calcula a fase experimental em graus a
partir do atraso (360 * f * deltaT)
```

```
phi=180*angle(H)/pi;            % Converte a fase do filtro FIR para graus
wlido = 2*pi*fr;                % Converte frequências medidas de Hz para
rad/s

% Comparação entre simulação e dados experimentais
figure(2)                       % Abre nova figura para plotagem

subplot(2,1,1), semilogx(f1,Hdb,wlido,hmed,'*r'), grid on
% Plota o ganho (dB) em escala logarítmica:
% - f1, Hdb: Resposta simulada do filtro FIR
% - wlido, hmed, '*r': Dados experimentais (marcados com asteriscos
vermelhos)

% Subplot da fase
subplot(2,1,2), semilogx(f1,phi,wlido,phimed,'*r'), grid on
% Plota a fase (graus) em escala logarítmica:
% - f1, phi: Resposta simulada do filtro FIR
% - wlido, phimed, '*r': Dados experimentais (marcados com asteriscos
vermelhos)
```

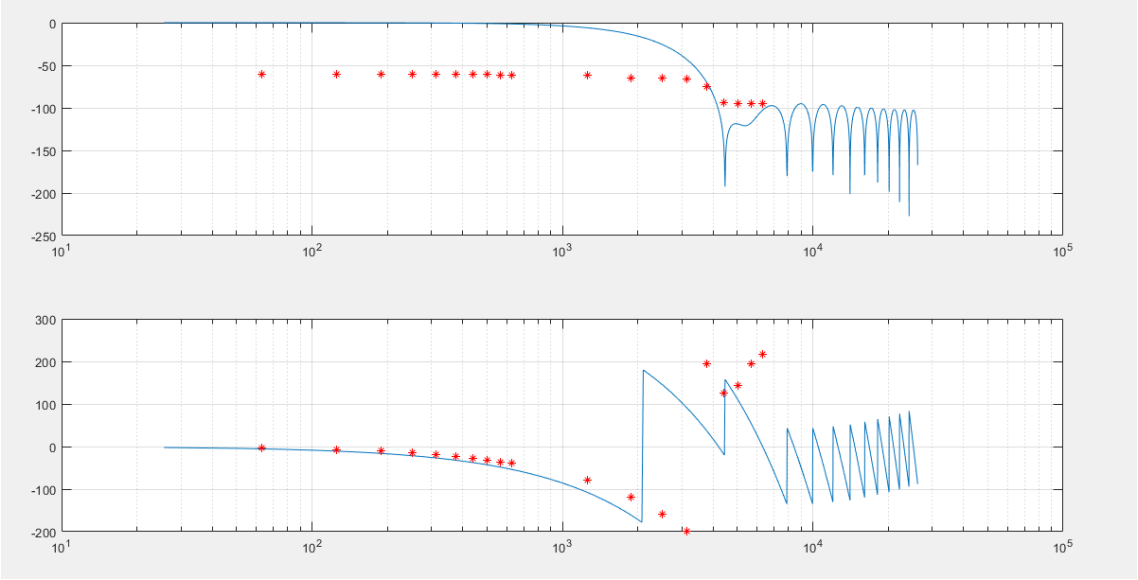

Figura 28-Imagem Obtida Matlab representação gráfica dos dados obtidos

Conclusão

O trabalho laboratorial teve como objetivo, implementar um filtro digital e para tal foi utilizado um conversor digital-analógico (DAC0832) e um amplificador operacional (LF356), integrado com um microcontrolador ATmega2560.Logo na primeira etapa de teste do ADC o Arduíno apresentou um defeito, que demorei algum tempo, a detetar pois a porta de entrada analógica usada PA0 (ADC0) não funcionava, de modo que tive de usar a porta ADC1 (pino PA1).
O circuito foi desenvolvido para realizar a conversão digital-analógica com o DAC0832, que foi controlado pelo ATmega2560. O DAC, corretamente configurado com os sinais de controlo e de dados, gerou uma saída em corrente que foi posteriormente convertida em tensão através do amplificador operacional LF356. Este último foi configurado em modo inversor para assegurar a conversão corrente-tensão necessária. A programação em Assembly permitiu o controlo eficiente do DAC, gerindo os sinais de entrada e saída, os valores armazenados e o processamento dos coeficientes do filtro e dos atrasos associados.

O filtro digital foi concebido em Matlab, definido um filtro FIR de ordem N=25 com uma frequência de corte normalizada de 0,002. Os coeficientes calculados foram escalados e arredondados para facilitar a implementação digital no hardware. Após o projeto, a resposta em frequência do filtro foi simulada no Matlab, obtendo-se uma previsão teórica do comportamento em termos de ganho e fase.

Durante a análise prática, verificou-se o comportamento do circuito com o auxílio do Osciloscópio (PicoScope7), confirmando que o sinal de entrada era corretamente processado e enviado para a saída, validando assim a comunicação entre o ATmega2560, o DAC0832 e o LF356. No entanto, ao comparar os dados experimentais com os resultados teóricos, surgiram discrepâncias, sobretudo a frequências mais elevadas. O ganho experimental apresentou uma atenuação superior à prevista, e a resposta em fase apresentou variações inesperadas.

As diferenças entre os resultados teóricos e práticos podem ser atribuídas a várias causas. Em primeiro lugar, distração na extração dos dados por minha parte, em segundo lugar devido a ruídos no ambiente de teste ou imprecisões nos componentes eletrónicos, podem ter afetado o desempenho do circuito. No entanto testei várias hipóteses, inclusive arredondei os valores de Vo às centésimas, mas sem sucesso, uma vez que as discrepâncias se mantiveram.

Apesar das discrepâncias, o trabalho demonstrou a funcionalidade básica do filtro projetado e a sua capacidade de processar sinais digitais e analógicos. Este trabalho prático permitiu compreender a importância de integrar o domínio teórico, representado pela simulação em Matlab, com a aplicação prática no circuito físico. A experiência reforçou também a necessidade de considerar as limitações práticas, como tolerâncias de componentes e ruídos, que impactam significativamente o desempenho de sistemas reais.

Em suma, este trabalho foi uma excelente oportunidade para aplicar conceitos de processamento de sinais, eletrónica analógica e programação a baixo nível. Embora os resultados não tenham sido totalmente alinhados com as expectativas teóricas, a experiência obtida será certamente valiosa para futuros projetos.